arXiv:2408.14354v1 [cs.SE] 26 Aug 2024

SWE-BENCH-JAVA: A GITHUB ISSUE RESOLVING BENCHMARK FOR JAVA

Daoguang Zan[1]* Zhirong Huang[1]* Ailun Yu[2]*
Shaoxin Lin[3] Yifan Shi[2] Wei Liu[2] Dong Chen[3] Zongshuai Qi[2]
Hao Yu [2] Lei Yu [1] Dezhi Ran[2] Muhan Zeng[3] Bo Shen[3] Pan Bian[3]
Guangtai Liang[3] Bei Guan[1] Pengjie Huang[4] Tao Xie[2] Yongji Wang[1] Qianxiang Wang[3]
[1]Chinese Academy of Science [2]Peking University [3]Huawei Co., Ltd. [4]Lingzhi-zhiguang Co., Ltd

https://multi-swe-bench.github.io

https://huggingface.co/datasets/Daoguang/Multi-SWE-bench

https://github.com/multi-swe-bench/multi-swe-bench-env

ABSTRACT

GitHub issue resolving is a critical task in software engineering, recently gaining significant attention in both industry and academia. Within this task, SWE-bench [1] has been released to evaluate issue resolving capabilities of large language models (LLMs), but has so far only focused on Python version. However, supporting more programming languages is also important, as there is a strong demand in industry. As a first step toward multilingual support, we have developed a Java version of SWE-bench, called `SWE-bench-java-verified`. We have publicly released the dataset, along with the corresponding Docker-based evaluation environment and leaderboard, which will be continuously maintained and updated in the coming months. To verify the reliability of `SWE-bench-java-verified`, we implement a classic method SWE-agent [2] and test several powerful LLMs on it. As is well known, developing a high-quality multi-lingual benchmark is time-consuming and labor-intensive, so we welcome contributions through pull requests or collaboration to accelerate its iteration and refinement, paving the way for fully automated programming.

1 Introduction

Automating software engineering tasks with large language models (LLMs) has gained considerable attention [3, 4, 5]. Beyond code generation, the issue resolving task proposed by SWE-bench [1] changes the role of LLMs from code assistants to fully autonomous AI programmers. SWE-bench contains $2,294$ issues from 12 widely-used open-sourced Python libraries. LLMs are tasked to generate a patch based on the issue description along with the buggy code repository. Within less than one year, the resolving rate on SWE-bench lite increased from 0.33% [1] (for RAG+GPT3.5) to 43.00% [6] (for CodeStory Aide+Mixed Models). SWE-bench lite is a subset of 300 issues selected from SWE-bench, chosen for their relatively clear descriptions and simple fix solutions.

SWE-bench [1] is centered on Python, which confined its evaluation to LLMs in Python-related fields such as data processing and artificial intelligence. However, it does not cover other common and necessary fields like web applications, mobile applications, and system programming, which rely on other programming languages [7, 8]. Therefore, as the first step in moving towards a multi-lingual issue resolving benchmark, we choose to develop a Java version of SWE-bench for the following two reasons:

1. **Popularity.** Java enjoys widespread popularity, making it one of the most widely adopted programming languages in the industry, particularly in fields like finance, cloud services, and Android application development. According to the TIOBE index[2] for August 2024, Java ranks among the top 4 languages, following Python, C, C++. With an active developer community, Java continues to grow, as evidenced by Oracle's data[3] showing 120 million developers worldwide, with over 1 million new developers added annually.

*Equal contribution

[2]`https://www.tiobe.com/tiobe-index`

[3]`https://blog.jetbrains.com/idea/2024/07/is-java-still-relevant-nowadays`

2. **Platform Independence.** Java programs run on a virtual machine, which automatically manages memory similar to Python, and compiles to Java byte-code that is interpreted by the virtual machine. While C and C++ are preferred for system programming due to their performance and memory efficiency, we believe that language models are not primarily designed to address performance issues. Therefore, we chose Java over the more performance-focused C/C++ as the first step.

This paper proposes a Java version SWE-bench, named `SWE-bench-java-verified`. We describe the details of dataset construction, the main challenges, and potential problems. With `SWE-bench-java-verified`, we also evaluate the performance of SWE-agent [2] with the state-of-the-art models including GPT-4o [9], GPT-4o-mini [10], DeepSeek-V2 [11], DeepSeekCoder-V2 [12], Doubao-pro [13]. Overall, the contributions of this paper are as follows:

- We meticulously created and manually verified the `SWE-bench-java-verified` benchmark, marking the first step in establishing a multilingual GitHub issue-resolving benchmark with a focus on Java. We plan to support more programming languages and make continuous improvements in the coming months. We also encourage the community to contribute by submitting pull requests to collaborate in advancing this field.
- We open-sourced the dataset, along with a comprehensive evaluation Docker environment and a leaderboard, to advance further research in this field.
- We implemented SWE-Agent [2] on `SWE-bench-java-verified` and derived several insightful findings that enhance our understanding of issue resolving in Java projects.

2 Multi-SWE-bench

2.1 Benchmark Construction

2.1.1 Workflow Overview

We construct `SWE-bench-java-verified` by following the work of SWE-bench [1]. Specifically, the workflow of constructing this benchmark comprises five phases:

1. **Candidate repository collection.** We collect candidate repositories for `SWE-bench-java-verified` construction from two sources: (1) Popular Java repositories on GitHub, which are collected by requesting a list of Java repositories sorted by their stars via GitHub API[4]; It is worth noting that we rule out repositories where Java is not the main language. (2) Repositories included in the Defects4j [7] database, which is a dataset collecting reproducible bugs across multiple Java repositories. As a result, we collect 53 repositories from GitHub and 17 repositories from Defect4J, obtaining a total of 70 candidate Java repositories. After careful manual selection and filtering, we narrowed down the list to 19 open-source Java repositories: 10 from source (1) and 9 from source (2).
2. **Issue instance crawling.** The issue instance crawling process was conducted in the following three steps: (1) We crawled all pull requests for the 19 selected repositories. (2) We filtered these pull requests by retaining only those that were associated with at least one issue and involved changes to test files. (3) For each pull request that met our selection criteria, we further crawled its detailed information, including "instance ID", "patch", "repository name", "base commit", "hints text", "creation date", "test patch", "issue statement", "environment setup commit", and "fail-to-pass". In total, we crawled 1,979 issue instances for 19 repositories.
3. **Runtime environment determination.** We determine the runtime environment of each issue through code reading and trial runs. To be specific, we determine the build tool, JDK version and compilation commands used in the target issue. Given the code repository associated with the target issue, we carry out three steps: (1) We identify the build tool type (maven[5] or Gradle[6]) from the repository structure; (2) We determine the JDK version used through reviewing the build configuration; (3) We compile the repository to establish its compilation commands. In this phase, we also filter out issues where the associated repository fails to be compiled, e.g. the repositories depending on additional development environments like Android SDK. As a result, we primarily derive a collection of 308 issue instances from the pool of 1,979, which are verified to compile successfully under the determined runtime environment.
4. **Fail-to-pass test extraction.** We extract fail-to-pass tests for each issue by comparing test results before and after applying the ground-truth patch. In detail, given an issue instance, we build two different containers

[4] `https://docs.github.com/en/rest`
[5] `https://maven.apache.org`
[6] `https://gradle.org`

to run the tests mentioned in the crawled test patch, respectively based on the code repository that has been patched with the fix and the code that hasn't. Then we parse the output test log to obtain the results of related tests, gathering those tests that fail before the issue fix but pass after it. Based on the test results, we also further conduct a filtering on issue instances. We only keep the issue instances which have at least one fail-to-pass test and no pass-to-fail test, deriving a set of 137 issue instances.

5. **Questionnaire-based manual verification.** To ensure the reliability of our benchmark in evaluating the LLMs' abilities in the issue resolving task, we conduct a comprehensive manual verification process. Following the recently published `SWE-bench-verified` annotation guidelines[7], we invite 10 software developers experienced in Java, to screen the above 137 issue instances. The verification process involved answering the following three questions of each issue: (Q1) the clarity of the issue description, rated on a scale from 0 to 3, where lower scores indicate greater clarity; (Q2) the comprehensiveness of test coverage in evaluating potential solutions, also rated from 0 to 3, where lower scores reflect better coverage; and (Q3) the presence of any major flaws that might necessitate exclusion from the dataset, with 0 indicating the absence of such flaws and 1 indicating their presence. Based on these annotations, we retained issues that satisfied the following criteria: "*(Q1 is 0 or Q1 is 1) and (Q2 is 0 or Q2 is 1) and (Q3 is 0)*". This stringent filtering resulted in a final dataset of 91 high-quality issue instances, covering 6 repositories.

2.1.2 Troubleshooting

During benchmark construction for `SWE-bench-java-verified`, we spot and fix a few issues which negatively affect the benchmark quality and evaluation efficiency. Although SWE-bench has established a complete and easy-to-use pipeline for mining issues and automatically evaluating solutions of issue resolving in Python projects, we still encountered some difficulties when migrating to Java. In this section, we will present the identified issues and discuss our solution for troubleshooting.

Base commit crawling errors in the original SWE-bench script. We found that the original SWE-bench script[8] sometimes crawls the incorrect base commit for pull requests, mainly because it indiscriminately uses the previous commit without considering branch differences. We have fixed this bug using "`git commit graph`" to distinguish between different branches, making the script more reliable and ready for use.

Redundant downloads of repositories and dependencies. We found it a burden to repeatedly download repositories and dependencies when running evaluations on different issue instances. To be specific, SWE-bench builds a separate docker container for each issue instance, where the corresponding repository will be pulled online to initialize the working directory; in addition, the dependencies will be installed to construct the runtime environment. However, some of the issues share the same code repository and their dependencies may overlap, which means redundant downloads occur. To reduce the redundancy of repository downloads, we pre-download all the selected repository locally all at once, and then replicate the downloaded repositories from local storage into containers. We also plan to similarly maintain a local cache for dependencies and directly mount it to the built container to avoid repeated downloads.

Possible compilation broken during incremental compilation. When pre-compiling dependencies to improve caching, applying incremental patches can sometimes cause compilation failures. Those failures may occur when the patches disrupt the project's build process, especially in projects with complex dependency structures. To address this, we carefully analyzed the source code and project architecture, identifying potential conflicts. We then crafted specific test commands tailored to each project's unique setup, ensuring that the build process remained stable despite the applied changes. This thorough process helped us maintain consistent build integrity across different environments.

2.2 Data Statistics

As illustrated in Figure 1, the `SWE-bench-java-verified` benchmark includes a total of 91 issues across 6 popular GitHub repositories. The distribution of issues varies among these repositories, with the highest concentration found in "`fasterxml/jackson-databind`" containing 49 issues, while "`apache/dubbo`" has the fewest, with only 4 issues. These 6 repositories span various domains, including data serialization (e.g., "`fasterxml/jackson-core`", "`fasterxml/jackson-dataformat-xml`"), web services (e.g., "`apache/dubbo`"), data formats (e.g., "`google/gson`"), and container tools (e.g., "`googlecontainertools/jib`"), demonstrating the dataset's broad coverage. This diversity underscores the dataset's representativeness, providing a wide-ranging testbed for evaluating LLMs' performance in automated issue resolving within the Java ecosystem.

[7] `https://cdn.openai.com/introducing-swe-bench-verified/swe-b-annotation-instructions.pdf`

[8] `https://github.com/princeton-nlp/SWE-bench/tree/main/swebench/collect`

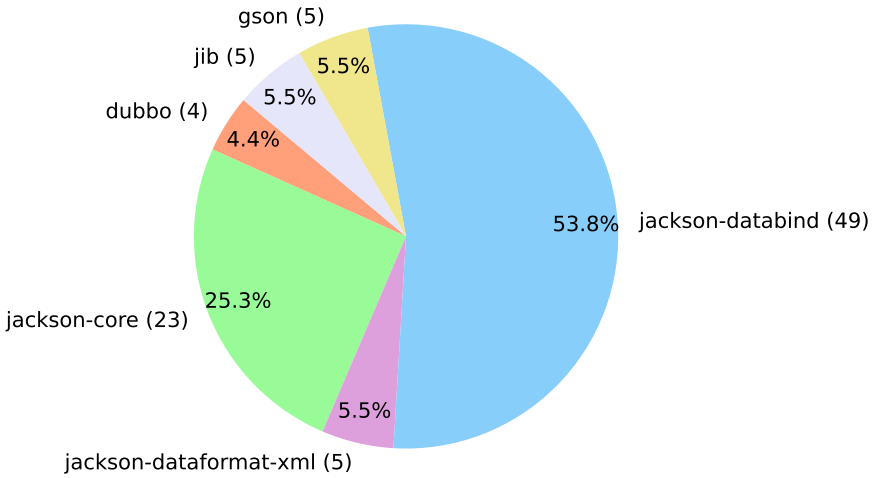

Figure 1: Distribution of `SWE-bench-java-verified` across 6 GitHub repositories.

Table 1: Summary statistics of `SWE-bench-java-verified`. "Repository #Files and #Lines" denote the total number of files and lines in the codebase, respectively. "Issue #Chars" indicates the average character count of issue descriptions. "Gold Patch # Files, # Lines and # Func." represents the average number of files, lines, and functions modified per patch in the repository, while "Test #Lines" denotes the total lines of code in the test cases.

User/Repository	**Star**	**Build Tool**	**Repository**		**Issue**	**Gold Patch**			**Test**
			# Files	**# Lines**	**# Chars**	**# Files**	**# Lines**	**# Func.**	**# Lines**
google/gson	23.2K	Maven	296	57.4K	1,511	9	89	23	88
FasterXML/jackson-core	2.2K	Maven	416	149.1K	1,241	58	125	230	110
FasterXML/jackson-databind	3.5K	Maven	1,259	303.2K	2,259	105	75	109	80
FasterXML/jackson-dataformat-xml	0.56K	Maven	220	277.3K	3,794	16	186	52	97
apache/dubbo	40.3K	Maven	4,387	457.3K	2,370	12	106	30	43
GoogleContainerTools/jib	13.6K	Gradle	1,195	102.7K	4,048	13	50	16	53
Mean	–	–	2,537	224.5K	2,537	36	105	77	79
Max	–	–	4,387	457.3K	4,048	105	186	230	110

Table 1 provides a summary of key statistics for the repositories included in the `SWE-bench-java-verified` dataset, also highlighting their diversity and representativeness. The repositories vary widely in popularity, as indicated by their star counts, ranging from 0.56K to 40.3K, showcasing a broad selection of Java projects from different domains. The repositories use either Maven or Gradle as build tools, with the majority being Maven projects. The repository sizes also differ significantly, with repository sizes ranging from 57.4K to 457.3K lines of code, and file counts from 220 to $4,387$. This variety provides a comprehensive testbed for evaluating the difficulty of patch generation. Besides, `SWE-bench-java-verified` often involves modifying multiple files (up to 105 in the largest case), a substantial number of lines (up to 186 lines), and numerous functions (up to 230 functions). Issues in these repositories vary in complexity, with character counts ranging from $1,241$ to $4,048$, and an average of $2,537$ characters, reflecting the detailed nature of the issue descriptions.

In summary, `SWE-bench-java-verified`'s broad star distribution, varied build tools, diverse repositories, and issue complexities make it a practical and reliable benchmark for evaluating LLMs in the context of Java-specific automated issue resolving and patch generation.

3 Experiments

3.1 Experimental Setup

3.1.1 Evaluation Metrics

Following SWE-bench [1], we adopt the Resolved Rate (%) as our evaluation metric. This metric indicates the proportion of issues in `SWE-bench-java-verified` that are successfully resolved. An issue is considered resolved

Table 2: Resolved rate (%) on `SWE-bench-java-verified` across various methods.

Methods	Resolved Rate (%)
SWE-agent+GPT-4o-2024-08-06	6.59% (6/91)
SWE-agent+GPT-4o-mini-2024-07-18	1.10% (1/91)
SWE-agent+DeepSeekCoder-V2-0724	7.69% (7/91)
SWE-agent+DeepSeek-V2-0628	9.89% (9/91)
SWE-agent+Doubao-pro-128k	1.10% (1/91)

only if all given test cases pass. This metric provides a precise measure of effectiveness in resolving real-world Java GitHub issues.

3.1.2 Evaluation Approaches and Models

To assess the reliability of `SWE-bench-java-verified`, we use SWE-agent [2], a classic approach that enhances LLM agents in software engineering tasks through a custom agent-computer interface (ACI). This interface empowers agents to autonomously create and edit code, navigate repositories, and execute tests. With its proven superior performance on benchmarks like SWE-bench [1], SWE-agent is well-suited for evaluating the robustness and practical value of our newly created `SWE-bench-java-verified`.

Note that we implemented SWE-agent rather hastily, without configuring the runtime environment for all Java issues in `SWE-bench-java-verified`. This will affect the agent's ability to accurately reproduce issues, potentially leading to lower results (Section 3.2) compared to SWE-agent's normal performance.

Based on SWE-agent, we employ several popular and powerful models, including GPT-4o-2024-08-06[9], GPT-4o-mini-2024-07-18, DeepSeek-Coder-V2-0724[10], DeepSeek-V2-0628, and Doubao-pro-128k[11].

3.2 Results

Table 2 demonstrates the varying capabilities of different models in solving `SWE-bench-java-verified` tasks. Compared to GPT and Doubao models, DeepSeek exhibits superior problem-solving abilities. The performance of GPT-4o significantly surpasses that of GPT-4o-mini, demonstrating the effectiveness of the model's comprehensive capabilities in problem-solving and highlighting the discriminative power of `SWE-bench-java-verified` in differentiating models. Overall, a greater amount of descriptive language in the issues leads to better problem-solving outcomes.

We observe that the more detailed the task description, the higher the requirement for the model's natural language understanding capability. As observed in Table 3, DeepSeek-V2 significantly outperforms DeepSeekCoder-V2 on the "`GoogleContainerTools/jib`" repository, which happens to have the most extensive natural language description among the six repositories (refer to Table1). Similar results are also reflected in the "`FasterXML/jackson-dataformat-xml`" repository. Conversely, in the "`FasterXML/jackson-core`" repository, which has the least textual content, DeepSeekCoder-V2 performs notably better than DeepSeek-V2. This further corroborates the positive correlation between the level of detail in task descriptions and the required natural language understanding ability of the models.

The diverse performance of different models across `SWE-bench-java-verified`'s various repositories underscores the diversity of this benchmark. Moreover, the significant performance disparities among models indicate that the dataset effectively distinguishes the capability differences between models, demonstrating good sensitivity and discrimination.

Furthermore, considering the scores achieved by the models, most tasks have not reached perfect or high scores, suggesting that `SWE-bench-java-verified` presents significant challenges to the work in related fields while also providing valuable guidance for future research. In summary, `SWE-bench-java-verified` reveals the limitations and strengths of models from multiple perspectives, holding great significance for advancing progress in relevant domains.

[9] `https://openai.com/api/pricing`

[10] `https://platform.deepseek.com/api-docs/zh-cn/pricing`

[11] `https://console.volcengine.com/ark/region:ark+cn-beijing/model/detail?Id=doubao-pro-128k`

Table 3: Proportion of issues resolved by each method across different repositories.

User/Repository	SWE-agent GPT-4o	SWE-agent GPT-4o-mini	SWE-agent DeepSeekCoder-V2	SWE-agent DeepSeek-V2	SWE-agent Doubao
google/gson	○	○	○	○	○
FasterXML/jackson-core	◔	○	◔	○	○
FasterXML/jackson-databind	◔	◔	◔	◔	◔
FasterXML/jackson-dataformat-xml	○	○	○	◔	○
apache/dubbo	○	○	○	○	○
GoogleContainerTools/jib	◔	○	○	◕	○

4 Related Works

The development of benchmarks for code generation has gained significant attention in recent years due to the increasing prominence of LLMs in programming tasks [3, 5, 14, 15, 16]. These benchmarks serve as critical tools for evaluating the capabilities of LLMs in understanding, generating, and refining code. Early efforts in this domain focused on primarily evaluating models in monolingual settings [17, 18, 19, 20, 21, 22]. For example, the HumanEval benchmark [19] offers a range of Python programming problems of varying difficulty to evaluate the functional correctness of code generated by LLMs. Similarly, the MBPP benchmark [20] is widely used to assess LLMs' performance on basic Python programming problems. As LLMs advanced, benchmarks became more sophisticated, evolving to better reflect real-world software engineering scenarios, such as library-oriented code generation, repository-level code completion, and issue resolving. In the field of library-oriented code generation, several Python-specific benchmarks have emerged, including PandasEval [23], NumpyEval [23], and TorchDataEval [24]. These benchmarks assess the ability of LLMs to generate code that interacts effectively with popular libraries. For repository-level code generation, early works like CoCoMIC [25] and RepoEval [26, 27] crafted datasets that require cross-file context to complete code, focusing exclusively on Python repositories. These benchmarks challenge LLMs to understand and complete code across multiple files within a repository. In the field of repository-level issue resolving, SWE-bench [28] was created, including $2,294$ software engineering problems derived from GitHub issues and corresponding pull requests across 12 Python repositories. Resolving issues in SWE-bench often demands a deep understanding of the repository and the ability to coordinate changes across multiple functions, classes, and files simultaneously [29, 30].

In addition to monolingual benchmarks, there has been a growing interest in multilingual benchmarks to assess the performance of LLMs across different programming languages. For example, Multilingual-HumanEval [31] and HumanEval-X [32] extends HumanEval [33] by providing equivalent tasks in multiple programming languages. Similarly, MBXP [34] extends the MBPP to include multilingual scenarios for more comprehensive evaluation. MultiPL-E [35] creates multilingual benchmarks based on HumanEval [33] and MBPP [36] in a translation-based way: it translates the two unit test-driven benchmarks to 18 additional programming languages. This shift towards multilingualism reflects the global nature of software development and the need for LLMs to perform well across various programming contexts. Moreover, recent work has emphasized the importance of benchmarking in real-world software engineering scenarios. The CoderEval [37] benchmark, for instance, is a context-aware benchmark for Java and Python that includes six levels of context dependency for code generation, such as class and file dependencies. Additionally, RepoBench [38] and CrossCodeEval [39] focus on more complex, real-world, multi-file programming tasks and thus serve as multilingual replacements of RepoEval [26].

Despite these advancements, aligning benchmarks with real-world programming needs remains challenging. SWE-bench [28] provides a benchmark for evaluation in a realistic software engineering setting for Python repositories, which is monolingual. To address this, we extend SWE-bench to include Java, creating a multilingual benchmark to better evaluate LLMs' coding abilities in real-world scenarios.

5 Conclusion and Future Works

This paper presents `SWE-bench-java-verified`, an evaluation benchmark specifically designed for resolving issues in Java projects. We detailed the construction process and conducted a comprehensive statistical analysis of the dataset. Additionally, we have open-sourced the dataset, evaluation Docker environment, and leaderboard. In the future, we plan to create benchmarks for more programming languages such as Go, Rust, C, and C++, while also continuing to improve the quality and coverage of the existing Java and Python datasets.

Acknowledgements

We express our deepest gratitude to the creators of the SWE-bench [1] dataset, whose foundational work our project is built upon. We thank Lingzhi-zhiguang Co., Ltd for providing the computing resources used in this research. This work was also supported by the National Key Research and Development Program of China under Grant No. 2022ZD0120201 - "Unified Representation and Knowledge Graph Construction for Science Popularization Resources".

References

[1] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[2] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[3] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. Large language models meet nl2code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7443–7464, 2023.

[4] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372*, 2023.

[5] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. A survey on language models for code. *arXiv preprint arXiv:2311.07989*, 2023.

[6] SANDEEP KUMAR PANI. Our sota multi-agent coding framework. *https://aide.dev/blog/sota-on-swe-bench-lite*, 2024.

[7] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ISSTA 2014, page 437–440, New York, NY, USA, 2014. Association for Computing Machinery.

[8] Yonghui Huang, Daniel Alencar da Costa, Feng Zhang, and Ying Zou. An empirical study on the issue reports with questions raised during the issue resolving process. *Empirical Software Engineering*, 24:718–750, 2019.

[9] OpenAI. Hello gpt-4o. 2024. `https://openai.com/index/hello-gpt-4o`.

[10] OpenAI. Gpt-4o mini: advancing cost-efficient intelligence. 2024. `https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence`.

[11] DeepSeek-AI, :, Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, Huazuo Gao, Kaige Gao, Wenjun Gao, Ruiqi Ge, Kang Guan, Daya Guo, Jianzhong Guo, Guangbo Hao, Zhewen Hao, Ying He, Wenjie Hu, Panpan Huang, Erhang Li, Guowei Li, Jiashi Li, Yao Li, Y. K. Li, Wenfeng Liang, Fangyun Lin, A. X. Liu, Bo Liu, Wen Liu, Xiaodong Liu, Xin Liu, Yiyuan Liu, Haoyu Lu, Shanghao Lu, Fuli Luo, Shirong Ma, Xiaotao Nie, Tian Pei, Yishi Piao, Junjie Qiu, Hui Qu, Tongzheng Ren, Zehui Ren, Chong Ruan, Zhangli Sha, Zhihong Shao, Junxiao Song, Xuecheng Su, Jingxiang Sun, Yaofeng Sun, Minghui Tang, Bingxuan Wang, Peiyi Wang, Shiyu Wang, Yaohui Wang, Yongji Wang, Tong Wu, Y. Wu, Xin Xie, Zhenda Xie, Ziwei Xie, Yiliang Xiong, Hanwei Xu, R. X. Xu, Yanhong Xu, Dejian Yang, Yuxiang You, Shuiping Yu, Xingkai Yu, B. Zhang, Haowei Zhang, Lecong Zhang, Liyue Zhang, Mingchuan Zhang, Minghua Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, Qihao Zhu, and Yuheng Zou. Deepseek llm: Scaling open-source language models with longtermism, 2024.

[12] DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bingxuan Wang, Junxiao Song, Deli Chen, Xin Xie, Kang Guan, Yuxiang You, Aixin Liu, Qiushi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence, 2024.

[13] Bytedance. Doubao. *https://console.volcengine.com/ark/region:ark+cn-beijing/model/detail?Id=doubao-pro-128k*, 2024.

[14] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*, 2023.

[15] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. PanGu-Coder2: Boosting Large Language Models for Code with Ranking Feedback, 2023.

[16] Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Wei Li, Yafen Yao, Yongshun Gong, Xiaolin Chen, Bei Guan, et al. CodeS: Natural Language to Code Repository via Multi-Layer Sketch. *arXiv preprint arXiv:2403.16443*, 2024.

[17] Miltiadis Allamanis and Charles Sutton. Mining source code repositories at massive scale using language modeling. In *2013 10th working conference on mining software repositories (MSR)*, pages 207–216. IEEE, 2013.

[18] Veselin Raychev, Pavol Bielik, and Martin Vechev. Probabilistic model for code with decision trees. *ACM SIGPLAN Notices*, 51(10):731–747, 2016.

[19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[20] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[21] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. Execution-based evaluation for open-domain code generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1271–1290, 2023.

[22] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. Mapping language to code in programmatic context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1643–1652, 2018.

[23] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. CERT: continual pre-training on sketches for library-oriented code generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 2369–2375, 2022.

[24] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Wang Yongji, and Jian-Guang Lou. When language model meets private library. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 277–288, 2022.

[25] Yangruibo Ding, Zijian Wang, Wasi U Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Cocomic: Code completion by jointly modeling in-file and cross-file context. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 3433–3445, 2024.

[26] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, 2023.

[27] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. GraphCoder: Enhancing Repository-Level Code Completion via Code Context Graph-based Retrieval and Language Model, 2024.

[28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024.

[29] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue Resolving with Multi-Agent and Task Graphs, 2024.

[30] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement, 2024.

[31] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. Multi-lingual evaluation of code generation models. In *ICLR*, 2023.

[32] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5673–5684, 2023.

[33] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[34] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. Multi-lingual evaluation of code generation models. In *The Eleventh International Conference on Learning Representations*.

[35] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. Multipl-e: a scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.

[36] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[37] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–12, 2024.

[38] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*.

[39] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems*, 36, 2024.

Author List and Contributions

Daoguang Zan: Project initiation, overall progress planning and management, experiment design, personnel coordination and task allocation, conceptual discussions, framework drafting, manuscript finalization, proofreading.

Zhirong Huang: Conceptual discussions, dataset collection and creation, data annotation, SWE-agent framework development, Section 2.1.1 (Benchmark Construction) writing, code writing and organization, server network debugging.

Ailun Yu: Conceptual discussions, dataset collection and creation, Docker evaluation environment configuration, data annotation, Section 2.1.1 (Benchmark Construction) framework and content writing, code writing and organization.

Shaoxin Lin: Conceptual discussions, data creation, data annotation.

Yifan Shi: Website development, including data visualization and continuous integration.

Wei Liu: Section 4 (Related Works) draft writing, data annotation.

Dong Chen: Section 1 (Introduction) draft writing, data annotation.

Zongshuai Qi: Section 3.2 (Results) draft writing, data annotation, results analysis, proofreading.

Hao Yu: Section 3.1 (Experimental Setup) draft writing, data annotation.

Lei Yu: Section 2.2 (Data Statistics) draft writing, data annotation, proofreading.

Dezhi Ran, **Muhan Zeng**: Data annotation, discussions.

Pengjie Huang: Initial idea discussions, computational resources provision.

Bo Shen, **Pan Bian**, **Guangtai Liang**, **Bei Guan**, **Tao Xie**, **Yongji Wang**: Guidance, discussions.

Qianxiang Wang: Project leadership.